

# Bases de Datos a Gran Escala

## Sesión 6 · Bases de datos de grafos: Neo4j

Diego Sevilla Ruiz

Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

dsevilla@um.es

# **Introducción a las bases de datos de grafos**

# Introducción a las BBDD de grafos

---

- Basadas en el concepto matemático de grafo
- **Nodos** o **vértices**, que contienen un identificador y un conjunto de propiedades
- Un conjunto de **arcos** o **aristas** que conectan pares de nodos, de forma dirigida o no; los arcos pueden tener sus propios atributos
- En el caso particular de Neo4j, los nodos también pueden tener un conjunto de etiquetas (**label**) y pares <clave, valor>. La etiqueta equivale a su «tipo»
- Incluyen un lenguaje de búsqueda de elementos y manipulación del grafo

# Ventajas de las BBDD de grafos

---

- **Navegación eficiente:** recorrido de relaciones en tiempo constante mediante *index-free adjacency*
- **Flexibilidad de esquema:** fácil añadir nuevos tipos de nodos y relaciones sin afectar a los datos existentes
- **Consultas intuitivas:** el lenguaje refleja la estructura visual del grafo (*ASCII art*)
- **Modelado natural:** muchos dominios son inherentemente grafos (redes sociales, dependencias, rutas)
- **Análisis de relaciones:** algoritmos especializados para detectar patrones, comunidades, rutas óptimas

# Desventajas y limitaciones

---

- **Operaciones agregadas masivas:** menos eficientes que los RDBMS para operaciones OLAP
- **Escalado horizontal limitado:** Neo4j Community sólo soporta una instancia; Enterprise usa replicación *master-slave*
- **Consultas globales costosas:** recorrer todo el grafo puede ser ineficiente
- **Curva de aprendizaje:** requiere pensar de forma diferente al modelo relacional
- **Tamaño de datos:** mejor rendimiento cuando el grafo completo cabe en memoria

# Casos de uso principales

---

- **Sistemas de recomendación:** productos, contenidos, amigos (Amazon, Netflix, LinkedIn)
- **Detección de fraude:** patrones sospechosos en transacciones financieras (PayPal, eBay)
- **Análisis de redes sociales:** influencia, comunidades, propagación de información
- **Gestión de identidad y acceso:** permisos, roles, jerarquías organizacionales
- **Análisis de dependencias:** software, cadenas de suministro, infraestructura IT
- **Gestión del conocimiento:** *knowledge graphs*, ontologías, grafos de conceptos
- **Redes de transporte:** rutas óptimas, análisis de tráfico

# Comparación con otros paradigmas

---

| Relaciones | JOIN s costosos           | Referencias limitadas | Nativas y eficientes |
|------------|---------------------------|-----------------------|----------------------|
| Esquema    | Rígido                    | Flexible              | Muy flexible         |
| Consultas  | SQL                       | <i>Query DSL</i>      | Cypher/Gremlin       |
| Mejor para | OLAP, agregaciones        | Documentos aislados   | Datos conectados     |
| Escalado   | Vertical/ <i>sharding</i> | Horizontal            | Limitado (Neo4j)     |

**Elegir grafos cuando:** las relaciones son tan importantes como los datos

# Property Graph vs. RDF

---

Dos modelos principales de bases de datos de grafos:

- **Property Graph** (Neo4j, Neptune, CosmosDB): nodos y relaciones con propiedades, relaciones con nombre y dirección, lenguajes Cypher/Gremlin — orientado a aplicaciones
- **RDF** (*Resource Description Framework*): tripletas sujeto-predicado-objeto, ontologías y razonamiento lógico, lenguaje SPARQL, bases de datos Blazegraph/GraphDB/Stardog — orientado a web semántica y conocimiento

Neo4j es *Property Graph*

# Tipos de BBDD de grafos

---

Se pueden clasificar atendiendo a dos dimensiones:

- **Almacenamiento subyacente:** nativo de grafos, o bien relacional/de otro tipo
- **Motor de procesamiento:** motor de grafos donde las entidades «apuntan» físicamente a otras (*index-free adjacency*), o bien se simula el procesamiento de grafos usando índices

**Ejemplos por cuadrante:** nativo/nativo — **Neo4j**, InfiniteGraph; no nativo/nativo — OrientDB, JanusGraph; no nativo/no nativo — GraphDB sobre RDBMS



# Aplicabilidad de grafos

---

- Los grafos, conceptualmente, aparecen en casi cualquier dominio
- Su flexibilidad hace que se puedan aplicar de diferentes formas:
  - Gestión de identidad y acceso (IAM): permisos, roles, herencia
  - Cadenas de suministro: proveedores, productos, rutas
  - Análisis de dependencias de software: paquetes, versiones
  - Redes de transporte: estaciones, rutas, conexiones
  - Estructuras jerárquicas recursivas: organizaciones, taxonomías

# Ejemplo: relación *follow* entre usuarios



Nótese cómo **Billy no ha seguido a Ruth**: las relaciones pueden ser unidireccionales o bidireccionales

# Guardando también los mensajes intercambiados





# Flexibilidad y eficiencia

---

Las bases de datos de grafos suplen dos carencias fundamentales:

1. La carencia expresiva del resto de paradigmas para ciertas relaciones entre datos
2. La eficiencia del tratamiento de estos datos con algoritmos basados en grafos, frente a otros paradigmas

**Clave de eficiencia:** *index-free adjacency* — cada nodo mantiene referencias directas a nodos adyacentes, sin búsquedas en índices, tiempo constante  $O(1)$  para acceder a vecinos

Ejemplo: una relación de amistad (*friend*) entre usuarios de una red social

# Una posible implementación relacional

| Person |        |
|--------|--------|
| ID     | Person |
| 1      | Alice  |
| 2      | Bob    |
| ...    | ...    |
| 99     | Zach   |



| PersonFriend |          |
|--------------|----------|
| PersonID     | FriendID |
| 1            | 2        |
| 2            | 1        |
| 2            | 99       |
| ...          | ...      |
| 99           | 1        |

# Consulta: amigos de Bob

---

```
SELECT p1.Person
FROM Person p1 JOIN PersonFriend
ON PersonFriend.FriendID = p1.ID
JOIN Person p2
ON PersonFriend.PersonID = p2.ID
WHERE p2.Person = 'Bob'
```

Es sencilla y no es computacionalmente muy compleja

# Consulta inversa

---

Como la relación de amistad no siempre es recíproca, a veces hay que hacer la búsqueda inversa:

```
SELECT p1.Person
FROM Person p1 JOIN PersonFriend
ON PersonFriend.PersonID = p1.ID
JOIN Person p2
ON PersonFriend.FriendID = p2.ID
WHERE p2.Person = 'Bob'
```

Esta consulta es más costosa que la anterior, porque hay que recorrer todas las filas de `PersonFriend`

# Amigos de los amigos de Alice

---

```
SELECT p1.Person AS PERSON, p2.Person AS FRIEND_OF_FRIEND
FROM PersonFriend pf1 JOIN Person p1
ON pf1.PersonID = p1.ID
JOIN PersonFriend pf2
ON pf2.PersonID = pf1.FriendID
JOIN Person p2
ON pf2.FriendID = p2.ID
WHERE p1.Person = 'Alice' AND pf2.FriendID <> p1.ID
```

Y sólo estamos bajando **un** nivel. Si bajáramos otro, se complicaría mucho más por los JOIN

# El experimento de *Neo4j in Action*

Partner y Vukotic compararon Neo4j y MySQL con esta tabla, buscando amigos de amigos en diferentes profundidades, en una red de 1 millón de personas con ~50 amigos cada una. (Experimento de 2014; el principio de eficiencia se mantiene en versiones modernas.)

| Prof. | Tiempos RDBMS | Tiempos Neo4j | Resultados |
|-------|---------------|---------------|------------|
| 2     | 0,016         | 0,01          | ~2 500     |
| 3     | 30,267        | 0,168         | ~110 000   |
| 4     | 1 543,505     | 1,359         | ~600 000   |

**Nota:** SQL puede ser mejor para operaciones masivas de agregación (OLAP) sobre todo el *dataset*



**Neo4j**

# Introducción a Neo4j

---

- Base de datos de grafos con posibilidad de extenderse a varios ordenadores (sólo uno soporta escritura, replicación *master-slave*)
- **Versiones:** Community (gratuita, una instancia, límites de autenticación), Enterprise (*clustering*, *backup* online, seguridad avanzada), Neo4j Aura (cloud gestionado, DBaaS)
- Compatible con el estándar Apache TinkerPop
- Lenguaje de creación y consulta de grafos: **Cypher**
- *Browser* web y consola para lanzar consultas; **Neo4j Desktop** para desarrollo local

# Introducción a Neo4j (ii)

---

- Multitud de algoritmos de recorrido de grafos, con la posibilidad de añadir más
- **APOC Library**: procedimientos y funciones avanzadas, muy utilizada
- **Graph Data Science (GDS)**: librería de algoritmos de grafos (PageRank, comunidades, etc.)
- Se puede usar desde Jupyter Notebook a través de una extensión
- **Drivers oficiales**: Python, Java, JavaScript, .NET, Go
- Ofrece también todos sus servicios a través de un API REST

# Grafos en Neo4j

---

- Grafos etiquetados y con propiedades, compuestos por **nodos**, **relaciones**, **propiedades** y **etiquetas**
- Los nodos contienen propiedades como pares clave-valor: las claves son cadenas, los valores tipos primitivos o *arrays*
- **Tipos de datos soportados:** String, Integer, Float, Boolean, Point, Date, DateTime, LocalTime, Duration, y listas de primitivos
- A los nodos se les puede etiquetar con una o más **etiquetas**, que los agrupan por rol dentro del grafo
- Las relaciones conectan nodos: **siempre tienen una dirección** (aunque se puede ignorar en consultas), un nombre propio, un nodo de inicio y otro de fin
- Como los nodos, las relaciones también tienen propiedades (p. ej. el peso o la calidad de un enlace), lo que además simplifica las consultas

# Modelo de datos visual en Neo4j

---



Nodos (círculos) con etiquetas y propiedades; relaciones dirigidas con tipo y propiedades opcionales

# Acceso desde Python

El *driver* oficial `neo4j` ofrece una API asíncrona sobre el protocolo *Bolt*:

```
from neo4j import AsyncDriver, AsyncGraphDatabase, EagerResult, basic_auth

cypher_query = """
MATCH (n)
RETURN n AS id
LIMIT 10;
"""

driver: AsyncDriver = AsyncGraphDatabase.driver(BOLT_URL, auth=basic_auth("neo4j", ""))

result: EagerResult = await driver.execute_query(cypher_query)
for record in result.records:
    print(record["id"])
```

`execute_query()` abre, ejecuta y cierra la transacción en un solo paso; para transacciones explícitas se usa `driver.session()`

# El lenguaje Cypher

# El lenguaje Cypher

---

Lenguaje de especificación de búsquedas y modificaciones en el grafo

Nodos:

```
( nombre:Etiqueta { propiedad: valor, ... } )
```

Relaciones, usando *ASCII art*:

```
(nodo_origen)-[:RELACIÓN]->(nodo_destino)
```

# Cypher: relaciones

---

Con propiedades:

```
(n1)-[:RELACIÓN {año: 2017}]->(n2)
```

En el otro sentido:

```
(nodo1)<-[:RELACIÓN]-(nodo2)
```

Omitiendo relaciones y nodos intermedios:

```
(nodo1)-[:RELACIÓN]->()-[:RELACIÓN2]->(nodo2)  
(nodo1)-->(nodo3)
```

# Cypher: repetición de relaciones

---

Repetida un número acotado o no acotado de veces:

```
(nodo1) - [:RELACIÓN*1..10] -> (nodo2)  
(nodo3) - [:RELACIÓN*0..] -> (nodoX)
```

Conjunto no limitado de relaciones, de forma general:

```
(nodo1) - [*] - (nodo2)
```

# Cypher: CREATE

---

Creación de nodos y enlaces:

```
CREATE (nodo1:Etiqueta1 { propX: valorX, ... } ),  
      (nodo2:Etiqueta2 { propY: valorY, ... } ),  
      ...  
      (nodo1) - [ :RELACIONADO_CON ] -> (nodo2)
```

# Consultas: MATCH

---

Consulta a través de ejemplos (*query by example*): especifica nodos y propiedades como ejemplo de lo que se busca

```
MATCH especificación, especificación, ...  
[WHERE especificación]  
[ORDER BY campo]  
[LIMIT x]  
RETURN [DISTINCT] nodos
```

MATCH también se usa para seleccionar nodos a crear ( CREATE ) o borrar ( DELETE / DETACH DELETE ). Retorna todos los *posts*:

```
MATCH (n:Post) RETURN n
```

# Consultas: restricciones

---

```
MATCH (n { nombre: 'Diego' }) RETURN n.edad;
```

En este caso se devuelve una tabla, no un subgrafo

# Consultas: nodos anónimos y caminos

---

Todos los abuelos con sus nietos:

```
MATCH (nieto), (abuelo),  
      (nieto)-[:HIJO_DE]->()-[:HIJO_DE]->(abuelo)  
RETURN abuelo, nieto
```

O incluso (nótense los nodos anónimos):

```
MATCH (nieto)-[:HIJO_DE]->()-[:HIJO_DE]->(abuelo)  
RETURN abuelo, nieto
```

También:

```
MATCH (nieto)-[:HIJO_DE*2]->(abuelo)  
RETURN abuelo, nieto
```

# Consultas con `OPTIONAL MATCH`

---

- `OPTIONAL MATCH` funciona como un `LEFT JOIN` en SQL
- Retorna `null` si no encuentra coincidencias

```
MATCH (p:Persona)
OPTIONAL MATCH (p)-[:TRABAJA_EN]->(c:Compañía)
RETURN p.nombre, c.nombre
```

Devuelve todas las personas, con su compañía si la tienen, `null` si no

# Funciones de agregación

---

Similares a SQL:

```
// Contar posts por usuario  
MATCH (u:User)-[:WROTE]->(p:Post)  
RETURN u.nombre, count(p) AS num_posts  
ORDER BY num_posts DESC
```

# Funciones de agregación (ii)

---

```
// Promedio de edad de usuarios  
MATCH (u:User)  
RETURN avg(u.edad) AS edad_promedio
```

```
// Recopilar amigos en una lista  
MATCH (p:Persona {nombre: 'Ana'})-[:AMIGO_DE]->(amigo)  
RETURN p.nombre, collect(amigo.nombre) AS amigos
```

Otras funciones: `sum()`, `max()`, `min()`, `collect()`,  
`percentileDisc()`

# Funciones de *strings*

---

```
MATCH (p:Post)
RETURN p.Title, toUpper(p.Title), toLower(p.Title)
```

```
MATCH (p:Post)
WHERE p.Title CONTAINS 'Neo4j'
RETURN p.Title
```

```
// split, substring, replace, trim
RETURN split("a,b,c", ",") AS lista
```

# Funciones de listas

---

```
MATCH (u:User)-[:AMIGO_DE]->(amigo)
WITH u, collect(amigo) AS amigos
RETURN u.nombre, size(amigos) AS num_amigos,
       head(amigos) AS primer_amigo
```

Otras: `tail()`, `last()`, `range()`, `reduce()`

# UNWIND: descomponer listas

---

UNWIND convierte una lista en filas individuales:

```
UNWIND [1, 2, 3] AS numero  
RETURN numero
```

Útil para procesar listas de datos:

```
WITH ['Alice', 'Bob', 'Charlie'] AS nombres  
UNWIND nombres AS nombre  
CREATE (:Persona {nombre: nombre})
```

## UNWIND con collect

---

```
MATCH (u:User)
WITH collect(u.email) AS emails
UNWIND emails AS email
RETURN toLower(email) AS email_normalizado
```

# Variables de caminos

---

Se pueden asignar caminos completos a variables:

```
MATCH p = (a:Persona)-[:AMIGO_DE*1..3]->(b:Persona)
WHERE a.nombre = 'Ana' AND b.nombre = 'Carlos'
RETURN p, length(p)
```

Útil para algoritmos de caminos:

```
MATCH p = shortestPath(
  (a:Persona {nombre: 'Ana'})-[*]-(b:Persona {nombre: 'Carlos'})
)
RETURN p, length(p) AS distancia
```

# Todos los caminos más cortos

---

```
MATCH p = allShortestPaths(  
  (a:Persona {nombre: 'Ana'})-[*]-(b:Persona {nombre: 'Carlos'})  
)  
RETURN p
```

# Filtros en relaciones

---

```
MATCH (u:User)-[r:AMIGO_DE WHERE r.desde > 2020]->(amigo)
RETURN u.nombre, amigo.nombre, r.desde
```

También en patrones de camino variable:

```
MATCH (a)-[r:CONOCE*1..3 WHERE r.confianza > 0.7]->(b)
RETURN a, b
```

# UNION: combinar resultados

---

UNION elimina duplicados; UNION ALL los mantiene

```
// Usuarios activos: escribieron post 0 comentaron  
MATCH (u:User)-[:WROTE]->(:Post)  
RETURN u.nombre AS usuario  
UNION  
MATCH (u:User)-[:COMMENTED]->(:Comment)  
RETURN u.nombre AS usuario
```

Las columnas devueltas deben coincidir en tipo y número

## UNION ALL : mantiene duplicados

---

```
MATCH (u:User)-[:WROTE]->(:Post)
RETURN u.nombre AS usuario, 'post' AS actividad
UNION ALL
MATCH (u:User)-[:COMMENTED]->(:Comment)
RETURN u.nombre AS usuario, 'comment' AS actividad
```

# CALL : procedimientos

---

- `CALL` ejecuta procedimientos almacenados
- Neo4j incluye muchos procedimientos *built-in*; APOC añade 450+ procedimientos más

```
// Listar todos los procedimientos
CALL dbms.procedures()
YIELD name, signature, description
WHERE name STARTS WITH 'apoc'
RETURN name, signature
LIMIT 10
```

# CALL : con y sin resultados

---

```
// Ejecutar procedimiento con resultados  
CALL db.labels() YIELD label  
RETURN label, count(*) AS count
```

```
// Procedimiento sin resultados (standalone)  
CALL db.clearQueryCaches()
```

# CALL : procedimientos útiles

---

```
// Ver esquema del grafo
CALL db.schema.visualization()

// Estadísticas de nodos
CALL db.stats.retrieve('NODES')

// Crear constraint dinámicamente con APOC
CALL apoc.schema.assert(
  {User: ['email']},
  {User: ['id']}
)
```

# Creación de nodos a partir de otros

---

Un patrón común: crear un hijo y añadirlo a alguien

```
MATCH (padre:Persona { nombre: 'Diego' })  
  CREATE (violeta:Persona {nombre: 'Violeta'}),  
    (violeta)-[:HIJO_DE]->(padre)
```

(También se puede usar `CREATE UNIQUE` o `MERGE` para evitar crear nodos nuevos si ya existen.)

# Creación de relaciones: **MERGE**

- Para crear sólo relaciones y usar los nodos que ya existan se suele utilizar **MERGE**
- **CREATE** no hace *match*: crea nodos conforme los necesita, y también la relación
- En la parte **MATCH** / **MERGE** se ligán los nodos y sólo se crean las relaciones (si alguna de mis hijas existe, no se crea):

```
MATCH (padre:Persona { nombre: 'Diego' })  
MERGE (violeta:Persona {nombre: 'Violeta'}),  
      (martina:Persona {nombre: 'Martina'}),  
      (violeta)-[:HIJO_DE]->(padre),  
      (martina)-[:HIJO_DE]->(padre)
```

# Creación de relaciones: **CREATE** tras **MATCH**

---

Si se sabe que los nodos ya existen, se incluyen en el **MATCH** y se usa **CREATE** sólo para las relaciones — mucho más rápido que **MERGE** :

```
MATCH (padre:Persona { nombre: 'Diego' })
      (violeta:Persona {nombre: 'Violeta'}),
      (martina:Persona {nombre: 'Martina'})
CREATE (violeta)-[:HIJO_DE]->(padre),
      (martina)-[:HIJO_DE]->(padre)
```

## Cláusula **WITH**

Extrae los datos a procesar posteriormente, dándoles opcionalmente un alias:

```
MATCH(a:Post) WITH count(a) as c RETURN c
```

Hay que especificar **todos** los valores que se pasan. Error: **b** no está definido:

```
MATCH(a:Post)-->(b) WITH a RETURN a,b LIMIT 100
```

Correcto: nodos *post* y su título en mayúsculas:

```
MATCH(a:Post) WITH a,toUpper(a.Title) AS t RETURN a,t LIMIT 100
```

# Índices

---

Se pueden crear índices para búsquedas rápidas sobre atributos:

```
CREATE INDEX post_title_idx IF NOT EXISTS  
FOR (n:Etiqueta) ON (n.nombre_campo)
```

# Índices y *constraints* avanzados

## Constraints de unicidad:

```
CREATE CONSTRAINT usuario_email_unique IF NOT EXISTS  
FOR (u:Usuario) REQUIRE u.email IS UNIQUE;
```

```
CREATE CONSTRAINT post_id_unique IF NOT EXISTS  
FOR (p:Post) REQUIRE p.id IS UNIQUE;
```

## Constraints de existencia (Enterprise):

```
CREATE CONSTRAINT usuario_id_exists IF NOT EXISTS  
FOR (u:Usuario) REQUIRE u.id IS NOT NULL;
```

## Índices compuestos (Enterprise):

```
CREATE INDEX usuario_nombre_apellido IF NOT EXISTS  
FOR (u:Usuario) ON (u.nombre, u.apellido);
```

# Índices *full-text*

---

```
CREATE FULLTEXT INDEX postContent IF NOT EXISTS  
FOR (n:Post) ON EACH [n.title, n.body];
```

```
// Uso  
CALL db.index.fulltext.queryNodes(  
    'postContent', 'Neo4j AND graphs'  
) YIELD node, score  
RETURN node.title, score
```

Ver índices y *constraints*:

```
SHOW INDEXES;  
SHOW CONSTRAINTS;
```

# Y consultas interesantes en Neo4j

---

CREATE

```
(NAmerica:Location {name:'North America', type:'continent'}),  
(USA:Location {name:'United States', type:'country' }),  
(Idaho:Location {name:'Idaho', type:'state' }),  
(Lucy:Person {name:'Lucy' }),  
(Idaho)-[:WITHIN]->(USA)-[:WITHIN]->(NAmerica),  
(Lucy) -[:BORN_IN]->(Idaho)
```

MATCH

```
(person)-[:BORN_IN]->()-[:WITHIN*0..]->(us:Location {name:'United States'}),  
(person)-[:LIVES_IN]->()-[:WITHIN*0..]->(eu:Location {name:'Europe'})  
RETURN person.name
```

# Modelado de datos en grafos

# Principios de modelado en grafos

---

- **Empezar por las preguntas**, no por los datos: ¿qué consultas necesitamos responder? Diseñar el modelo para optimizarlas
- ***Whiteboard-friendly***: dibujar el modelo en una pizarra
- **Principio**: los datos conectados son tan importantes como los datos mismos
- **Desnormalizar**: a diferencia de RDBMS, duplicar datos puede ser beneficioso
- **Evolución**: el modelo puede evolucionar sin migraciones complejas

# Nodos vs. propiedades

---

## Usar nodos cuando:

- El elemento tiene identidad propia, hay que buscar por él o tiene relaciones con otros elementos
- Ejemplo: Usuario, Producto, Ciudad

## Usar propiedades cuando:

- Son atributos simples del nodo, sin búsquedas frecuentes ni relaciones propias
- Ejemplo: nombre, edad, precio, fecha

**Ejemplo:** ciudad como nodo si se necesita «todas las personas en Madrid»; como propiedad si sólo se guarda el lugar de nacimiento sin más consultas

# Relaciones vs. propiedades

---

**Usar relaciones cuando:** conectan dos entidades, la conexión tiene semántica importante, hay que navegarla, o puede tener propiedades (peso, fecha, tipo)

**Usar propiedades cuando:** el valor es un dato simple del nodo, sin necesidad de navegación

**Ejemplo:** «Trabaja en» como relación si se quiere «empleados de la empresa X»; como propiedad si sólo se guarda el nombre, sin navegación

# Modelado de relaciones n-arias

---

- Neo4j sólo soporta relaciones binarias (entre dos nodos)
- Para relaciones ternarias o n-arias: **usar nodo intermedio**

```
// Mal diseño: no se puede representar directamente
(Actor)-[:ACTUA_EN {rol: 'Batman'}]->(Película)

// Buen diseño: nodo intermedio
(Actor)-[:ACTUO]->(Actuación)-[:EN_PELÍCULA]->(Película)
// Actuación tiene propiedades: rol, año, caché
```

Ventaja: podemos añadir más información a la actuación

# Modelado temporal: *Time-Tree Pattern*

---

Para eventos temporales, crear un árbol de tiempo permite consultas eficientes por rangos de fechas:

```
// Estructura: Year -> Month -> Day
(y:Year {valor: 2025})-[:HAS_MONTH]->(m:Month {valor: 11})-[:HAS_DAY]->(d:Day {valor: 10})

// Eventos conectados a días
(evento:Evento)-[:OCURRIO_EN]->(d)

// Consulta: eventos en noviembre 2025
MATCH (y:Year {valor: 2025})-[:HAS_MONTH]->(m:Month {valor: 11})
      -[:HAS_DAY]->(d:Day {valor: 10}),
      (evento:Evento)-[:OCURRIO_EN]->(d)
RETURN evento
```

# ***Fan-out y fan-in***

---

- **Fan-out:** un nodo con muchas relaciones salientes (p. ej. usuario con miles de posts) — puede ser lento al cargar el nodo
- **Fan-in:** un nodo con muchas relaciones entrantes (p. ej. *tag* popular en miles de posts) — problema de rendimiento similar
- **Solución:** usar nodos intermedios o diferentes tipos de relaciones
  - Agrupar por fecha, categoría, etc
  - Ejemplo: Usuario → Año → Mes → Posts

# Algoritmos de grafos

# Graph Data Science Library (GDS)

---

- Librería de Neo4j para algoritmos de grafos avanzados; se instala como *plugin* separado
- Tres categorías principales:
  - i. Pathfinding:** encontrar caminos y rutas óptimas
  - ii. Centralidad:** identificar nodos importantes
  - iii. Detección de comunidades:** encontrar grupos conectados
  - iv. Similitud:** encontrar nodos similares
- Operación típica: crear proyección del grafo en memoria, ejecutar el algoritmo, y escribir o analizar los resultados

# Pathfinding: *shortest path*

---

Camino más corto entre dos nodos:

```
MATCH p = shortestPath(  
  (a:Ciudad {nombre: 'Madrid'})-[*]-(b:Ciudad {nombre: 'Barcelona'})  
)  
RETURN p, length(p)
```

# Pathfinding: Dijkstra

---

Camino más corto con pesos:

```
MATCH (a:Ciudad {nombre: 'Madrid'}), (b:Ciudad {nombre: 'Barcelona'})
CALL gds.shortestPath.dijkstra.stream({
  sourceNode: a, targetNode: b,
  relationshipWeightProperty: 'distancia'
})
YIELD path, totalCost
RETURN path, totalCost
```

- **A\***: camino con heurística (más rápido)
- **All Shortest Paths**: todos los caminos más cortos

# Algoritmos de centralidad

---

**PageRank:** importancia basada en conexiones — original de Google, identifica nodos influyentes

```
CALL gds.pageRank.stream({  
  nodeProjection: 'User',  
  relationshipProjection: 'FOLLOWS'  
})  
YIELD nodeId, score  
RETURN gds.util.asNode(nodeId).nombre AS usuario, score  
ORDER BY score DESC LIMIT 10
```

- **Betweenness Centrality:** nodos que actúan como puentes
- **Closeness Centrality:** nodos más cerca del resto
- **Degree Centrality:** basado en número de conexiones

# Algoritmos de comunidades

---

**Louvain:** detecta comunidades optimizando modularidad

```
CALL gds.louvain.stream({  
  nodeProjection: 'User',  
  relationshipProjection: 'FRIENDS_WITH'  
})  
YIELD nodeId, communityId  
RETURN communityId, count(*) AS size  
ORDER BY size DESC
```

- **Label Propagation:** propagación de etiquetas para comunidades
- **Weakly Connected Components:** componentes conectados
- **Triangle Count:** detecta *clustering*

# Algoritmos de similitud

**Node Similarity:** nodos con vecinos similares

```
CALL gds.nodeSimilarity.stream({
  nodeProjection: 'User',
  relationshipProjection: 'LIKES'
})
YIELD node1, node2, similarity
WHERE similarity > 0.5
RETURN gds.util.asNode(node1).nombre,
       gds.util.asNode(node2).nombre,
       similarity
```

- **Jaccard Similarity:** basado en intersección de vecinos
- **Cosine Similarity:** para vectores de propiedades — útil para sistemas de recomendación

# **Rendimiento y buenas prácticas**

# Evitar productos cartesianos

---

```
// MAL - Cartesian product
MATCH (u:User), (p:Post)
WHERE u.id = p.userId
RETURN u, p

// BIEN - Usar relaciones
MATCH (u:User)-[:WROTE]->(p:Post)
RETURN u, p
```

# Usar índices

---

```
// Crear índice primero
CREATE INDEX user_email IF NOT EXISTS
FOR (u:User) ON (u.email);

// Consulta aprovecha el índice
MATCH (u:User {email: 'user@example.com'})
RETURN u
```

# Usar parámetros

---

Para reutilización de planes:

```
// Sin parámetros - nuevo plan cada vez
MATCH (u:User {email: 'user1@example.com'}) RETURN u
MATCH (u:User {email: 'user2@example.com'}) RETURN u

// Con parámetros - plan reutilizado
MATCH (u:User {email: $email}) RETURN u
```

# Limitar temprano

---

```
// Limitar después de procesar todo
MATCH (u:User)-[:WROTE]->(p:Post)
RETURN u, count(p) AS posts
ORDER BY posts DESC
LIMIT 10

// Mejor: limitar antes si es posible
MATCH (u:User)
WITH u LIMIT 1000
MATCH (u)-[:WROTE]->(p:Post)
RETURN u, count(p) AS posts
```

# Analizar rendimiento: EXPLAIN

---

Muestra el plan de ejecución sin ejecutar:

```
EXPLAIN  
MATCH (u:User {email: $email})-[:WROTE]->(p:Post)  
RETURN u, p
```

Útil para ver si usa índices

# Analizar rendimiento: PROFILE

---

Ejecuta y muestra estadísticas detalladas:

```
PROFILE  
MATCH (u:User {email: $email})-[:WROTE]->(p:Post)  
RETURN u, p
```

Muestra: filas procesadas en cada paso, *db hits* (accesos a disco/caché) y tiempo de ejecución

Buscar: alto *db hits*, muchas filas procesadas innecesariamente

# Operaciones masivas: *batching*

---

Para operaciones masivas, usar transacciones por lotes; no insertar/actualizar millones de nodos en una sola transacción:

```
// Cargar CSV en lotes de 1000
LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row
CALL {
  WITH row
  MERGE (u:User {id: row.id})
  SET u.nombre = row.nombre, u.email = row.email
} IN TRANSACTIONS OF 1000 ROWS
```

# APOC para procesamiento paralelo

---

```
CALL apoc.periodic.iterate(  
  "MATCH (u:User) RETURN u",  
  "SET u.processed = true",  
  {batchSize: 1000, parallel: true}  
)
```

# Configuración de memoria

---

- **Heap memory:** para transacciones y consultas. Típico 8–16 GB en producción; se configura con `dbms.memory.heap.max_size` en `neo4j.conf`
- **Page cache:** para cachear el grafo en memoria. Lo más grande posible (RAM disponible – *heap* – SO); se configura con `dbms.memory.pagecache.size`. Idealmente, todo el grafo en *page cache*
- **Regla general:** 50 % de la RAM para *page cache*, 25 % para *heap*
- **Monitorizar:** `:queries` en el *browser*, métricas JMX

# Transacciones y concurrencia

---

- Neo4j es **ACID** completo
- Usa ***write locks*** en nodos/relaciones modificadas; los ***read locks*** no bloquean otras lecturas
- **Deadlocks** posibles en escrituras concurrentes: Neo4j los detecta y aborta la transacción automáticamente — hay que reintentarla
- **Buenas prácticas:** transacciones cortas, procesar en lotes (*batching*), usar `MERGE` con cuidado (puede causar conflictos), evitar modificar los mismos nodos desde múltiples *threads*

# **Comparación con SQL para Stack Overflow**

# Modelo de datos de Stack Overflow

---

**Entidades principales:** Users , Posts (preguntas y respuestas), Tags , Comments

## Relaciones:

- User -[:WROTE]-> Post
- Post -[:ANSWERS]-> Post (respuesta a pregunta)
- Post -[:TAGGED\_WITH]-> Tag
- User -[:COMMENTED]-> Comment -[:ON]-> Post

# Modelo visual de Stack Overflow

---

```
( :User ) - [ :WROTE ] -> ( :Post:Question )
      |
      [ :TAGGED_WITH ]
      |
      v
      ( :Tag )
      ^
      |
      [ :TAGS ]
      |
( :User ) - [ :WROTE ] -> ( :Post:Answer ) - [ :ANSWERS ] -> ( :Post:Question )
```

# Stack Overflow, SQL

---

RQ4, sesión 2: parejas de usuarios que se han respondido mutuamente

```
CREATE VIEW Pairs AS SELECT DISTINCT Q.OwnerUserId AS questioner, A.OwnerUserId AS answerer
FROM Posts Q JOIN Posts A ON Q.AcceptedAnswerId = A.Id
AND Q.OwnerUserId <> A.OwnerUserId;

SELECT P1.questioner AS U1, P2.questioner AS U2
FROM Pairs P1, Pairs P2
WHERE P1.questioner = P2.answerer
AND P2.questioner = P1.answerer
AND P1.questioner < P2.questioner;
```

# Stack Overflow, SQL (con CTE)

---

```
WITH Pairs AS (  
    SELECT DISTINCT Q.OwnerUserId AS questioner, A.OwnerUserId AS answerer  
    FROM Posts Q JOIN Posts A ON Q.AcceptedAnswerId = A.Id  
    AND Q.OwnerUserId <> A.OwnerUserId  
)  
SELECT P1.questioner AS U1, P2.questioner AS U2  
FROM Pairs P1, Pairs P2  
WHERE P1.questioner = P2.answerer  
    AND P2.questioner = P1.answerer  
    AND P1.questioner < P2.questioner;
```

# Stack Overflow, Neo4j

## Consulta: parejas recíprocas

```
MATCH
    (u:User) - [:WROTE] ->() - [:ANSWERS] ->() <- [:WROTE] - (u2),
    (u2:User) - [:WROTE] ->() - [:ANSWERS] ->() <- [:WROTE] - (u)
WHERE toInt(u.Id) < toInt(u2.Id)
RETURN DISTINCT u.Id, u2.Id;
```

## Crear relación **RECIPROCATE**

```
MATCH
    (u:User) - [:WROTE] ->() - [:ANSWERS] ->() <- [:WROTE] - (u2),
    (u2:User) - [:WROTE] ->() - [:ANSWERS] ->() <- [:WROTE] - (u)
WHERE toInt(u.Id) < toInt(u2.Id)
MERGE (u2) - [:RECIPROCATE] -> (u)
MERGE (u) - [:RECIPROCATE] -> (u2);
```

# Top usuarios por respuestas aceptadas

---

```
MATCH (u:User)-[:WROTE]->(a:Answer)-[:ANSWERS]->(q:Question)
WHERE q.AcceptedAnswerId = a.Id
RETURN u.DisplayName, count(a) AS respuestas_aceptadas
ORDER BY respuestas_aceptadas DESC
LIMIT 10
```

# Preguntas sin respuesta

---

```
MATCH (q:Question)
WHERE NOT (q)<-[ :ANSWERS ]-()
RETURN q.Title, q.CreationDate
ORDER BY q.ViewCount DESC
LIMIT 100
```

# Tags más relacionados (co-ocurrencia)

---

```
MATCH (t1:Tag)<-[:TAGGED_WITH]-(q:Question)-[:TAGGED_WITH]->(t2:Tag)
WHERE t1.Name < t2.Name
RETURN t1.Name, t2.Name, count(q) AS veces
ORDER BY veces DESC
LIMIT 20
```

# Expertos por *tag*

---

Más respuestas aceptadas en un *tag*:

```
MATCH (u:User)-[:WROTE]->(a:Answer)-[:ANSWERS]->(q:Question)
      -[:TAGGED_WITH]->(t:Tag {Name: 'neo4j'})
WHERE q.AcceptedAnswerId = a.Id
RETURN u.DisplayName, count(a) AS respuestas
ORDER BY respuestas DESC
LIMIT 10
```

# Sistema de recomendación simple

---

Recomendar amigos: amigos de mis amigos que no conozco

```
MATCH (yo:User {nombre: 'Ana'})-[:AMIGO_DE]->()-[:AMIGO_DE]->(recomendado)
WHERE NOT (yo)-[:AMIGO_DE]->(recomendado) AND recomendado <> yo
RETURN recomendado.nombre, count(*) AS amigos_comunes
ORDER BY amigos_comunes DESC
LIMIT 10
```

# Recomendar productos

---

Basado en compras similares:

```
MATCH (yo:User {id: 123})-[:COMPRO]->(p:Producto)
      <-[:COMPRO]-(otro:User)-[:COMPRO]->(recomendacion:Producto)
WHERE NOT (yo)-[:COMPRO]->(recomendacion)
RETURN recomendacion.nombre, count(*) AS popularidad
ORDER BY popularidad DESC
LIMIT 10
```

# Recomendar contenido

---

Por similitud de intereses:

```
MATCH (yo:User {id: 123})-[:INTERESTED_IN]->(t:Tag)
      <-[:INTERESTED_IN]-(similar:User)-[:WROTE]->(post:Post)
WHERE NOT (yo)-[:WROTE]->(post)
WITH post, count(DISTINCT similar) AS usuarios_similares
RETURN post.titulo, usuarios_similares
ORDER BY usuarios_similares DESC
LIMIT 10
```

# Carga de datos desde CSV

---

- Neo4j puede cargar datos directamente desde archivos CSV
- Los archivos deben estar en `import/`, dentro del directorio de Neo4j

Cargar usuarios:

```
LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row
CALL {
  WITH row
  MERGE (u:User {Id: row.Id})
  SET u.DisplayName = row.DisplayName,
      u.Reputation = toInteger(row.Reputation),
      u.CreationDate = datetime(row.CreationDate)
} IN TRANSACTIONS OF 1000 ROWS
```

# Cargar *posts* y relaciones

```
LOAD CSV WITH HEADERS FROM 'file:///posts.csv' AS row
CALL {
  WITH row
  MERGE (p:Post {Id: row.Id})
  SET p.Title = row.Title,
      p.Body = row.Body,
      p.PostTypeId = toInteger(row.PostTypeId)

  // Crear relación con usuario
  WITH p, row
  MATCH (u:User {Id: row.OwnerUserId})
  MERGE (u)-[:WROTE]->(p)

  // Si es respuesta, crear relación ANSWERS
  WITH p, row
  WHERE row.ParentId IS NOT NULL
  MATCH (q:Post {Id: row.ParentId})
  MERGE (p)-[:ANSWERS]->(q)
} IN TRANSACTIONS OF 500 ROWS
```

# Importación masiva

---

Para grandes volúmenes, `neo4j-admin import`:

```
neo4j-admin import \  
  --nodes=User=users.csv \  
  --nodes=Post=posts.csv \  
  --relationships=WROTE=wrote.csv \  
  --relationships=ANSWERS=answers.csv
```

Mucho más rápido (millones de nodos por minuto). Requiere base de datos vacía

# Recursos y herramientas

# Recursos de aprendizaje

---

- **GraphAcademy** ([graphacademy.neo4j.com](https://graphacademy.neo4j.com)): cursos gratuitos online interactivos y certificaciones oficiales, de básico a avanzado
- **Neo4j Sandbox** ([sandbox.neo4j.com](https://sandbox.neo4j.com)): entornos precargados para experimentar (Movie Graph, Twitter, Game of Thrones, etc.), sin instalación
- **Documentación oficial** ([neo4j.com/docs](https://neo4j.com/docs))
- **Comunidad:** Neo4j Community Forum, Stack Overflow (*tag* `neo4j` )
- **Canal de YouTube** de Neo4j: tutoriales y casos de uso

# Herramientas y extensiones

---

- **Neo4j Desktop:** entorno de desarrollo local, gestión de varias bases de datos, *plugins* (APOC, GDS, Bloom)
- **Neo4j Bloom:** visualización y exploración de grafos, interfaz para usuarios no técnicos, búsqueda en lenguaje natural
- **APOC** (*Awesome Procedures On Cypher*): 450+ procedimientos y funciones; importación/exportación de datos; utilidades de texto, fecha, JSON
- **Neoclipse:** IDE Eclipse para Neo4j (descontinuado, pero útil)